



PUCE
Pontificia Universidad
Católica del Ecuador

PUCE VIRTUAL

Facultad de Ingeniería
Ingeniería en Ciberseguridad

INTELIGENCIA ARTIFICIAL

Tarea — Semana 3

**Optimización de un detector de anomalías
para un centro de operaciones de seguridad**

Isolation Forest · Precision, Recall, F1-score y carga operacional

Autor

Alejandro David Carriel Zambrano

Semilla 42 · N = 1000 conexiones · scikit-learn

Quito — Ecuador · 2026

1. De qué va el problema

El caso es bastante directo: un SOC (centro de operaciones de seguridad) tiene un detector de anomalías para priorizar conexiones de red que se ven raras. El equipo no da abasto con todo, así que como tope pueden revisar **60 alertas por cada 1000 conexiones**. La dirección quiere cachar la mayor cantidad de amenazas posible, pero tampoco quiere ahogar a los analistas con falsas alarmas. O sea, el problema no es solo “qué tan bueno es el modelo”, sino elegir un umbral que no se pase de listo ni se quede corto.

Isolation Forest (Liu et al., 2008) no te dice “esto es un ataque” o “esto es tráfico normal”. Lo que hace es aislar observaciones que se cortan fácil con pocos splits aleatorios, o sea, puntos estadísticamente raros. El parámetro **contamination** es el que decide qué porcentaje de puntos se van a marcar como anómalos y, en la práctica, cuántos tickets le caen al SOC.

2. Cómo lo armé

Partí del laboratorio guiado. Simulé 950 conexiones normales y 50 anómalas (tasa real = 0,05) con cinco variables: duración en segundos, bytes enviados, paquetes, intentos fallidos y puertos contactados. Las anomalías las generé con medias bien corridas (más duración, más volumen, más fallos y más puertos). Ojo: las etiquetas reales **no se las pasé al modelo**; solo las usé después para evaluar. Las variables las estandaricé con StandardScaler.

Entrené Isolation Forest con **n_estimators = 200** y **random_state = 42** (Pedregosa et al., 2011). Probé sí o sí estos valores de contamination: 0,01; 0,03; 0,05; 0,06; 0,08; 0,10 y 0,12. En cada uno calculé alertas, verdaderos positivos (VP), falsos positivos (FP), falsos negativos (FN), Precision, Recall y F1-score. También comparé la config recomendada contra una variante con n_estimators = 100, dejando contamination = 0,05. Como en 0,05 no hubo FP ni FN, revisé los errores en las configs vecinas: 0,03 (ahí sí hay FN) y 0,08 (ahí hay FP), que es lo que pide la rúbrica cuando “existan”.

3. Tabla de resultados

contamination	Alertas	VP	FP	FN	Precision	Recall	F1-score
0.01	10	10	0	40	1.000	0.200	0.333
0.03	30	30	0	20	1.000	0.600	0.750
0.05	50	50	0	0	1.000	1.000	1.000
0.06	60	50	10	0	0.833	1.000	0.909
0.08	80	50	30	0	0.625	1.000	0.769
0.10	100	50	50	0	0.500	1.000	0.667
0.12	120	50	70	0	0.417	1.000	0.588

Tabla 1. Desempeño según contamination (n_estimators = 200). La fila verde es la que recomiendo. En 0,05 el laboratorio salió con 0 FP y 0 FN.

4. Gráficos

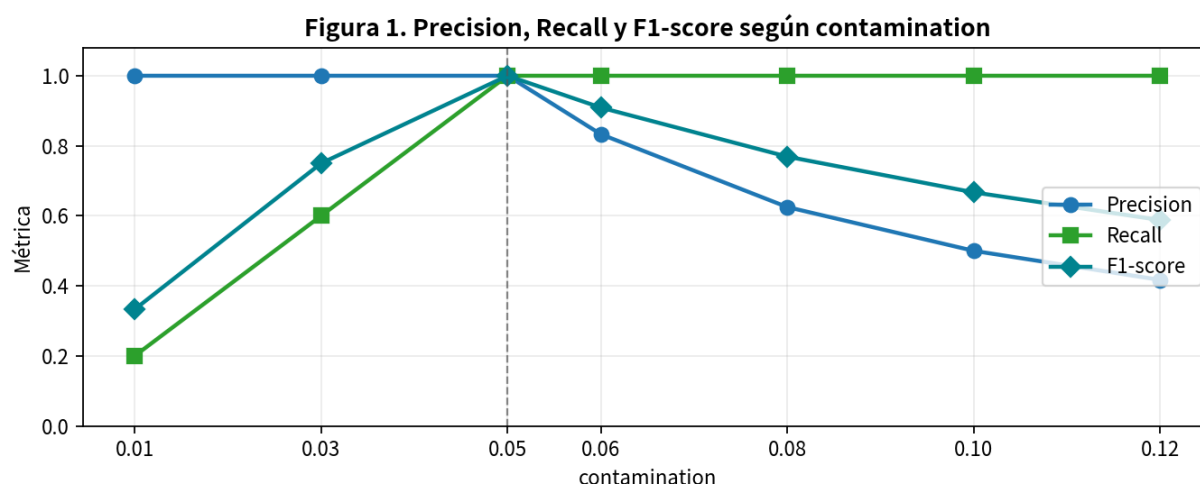


Figura 1. El F1 más alto cae en contaminación = 0,05. De ahí para arriba el Recall se queda en 1 y la Precision se empieza a caer.

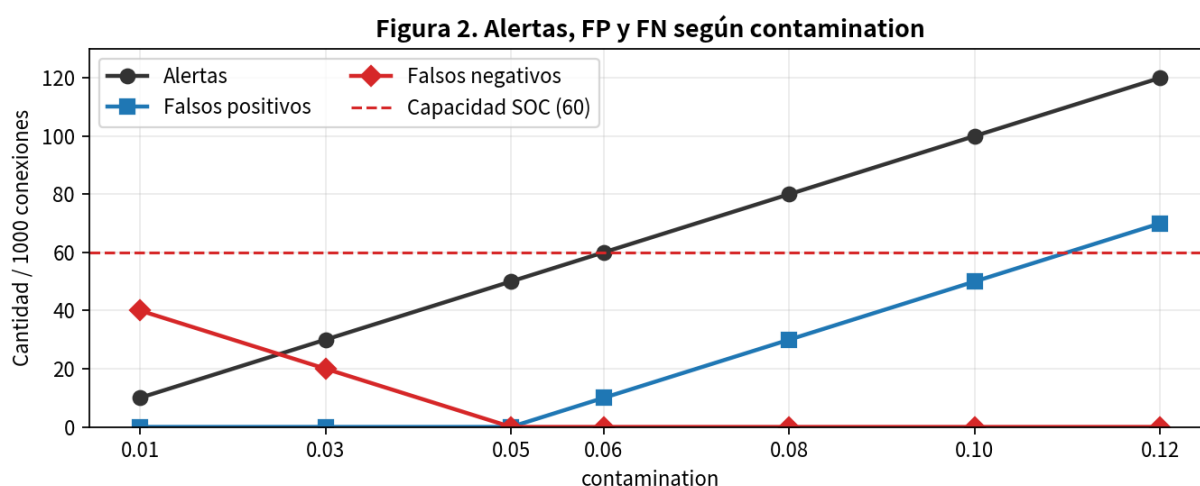


Figura 2. La línea roja punteada es el tope del SOC. 0,08; 0,10 y 0,12 ya se pasan de las 60 alertas.

5. Configuración que me quedo

Me quedo con Isolation Forest, **n_estimators = 200** y **contamination = 0,05**. Te genera 50 alertas por cada 1000 conexiones (cabe dentro de las 60), y en este set simulado sale Precision = 1,000, Recall = 1,000 y F1 = 1,000. Cero FP y cero FN. La de 0,06 usa el cupo completo (60 alertas) y solo mete 10 falsos positivos, sin cachar amenazas extra. Bajar n_estimators a 100 no mejora el F1: con las clases tan separadas, el que manda en la carga del SOC es contaminación, no cuántos árboles le pongas.

6. Preguntas obligatorias

1. ¿Qué configuración obtuvo el mayor F1-score?

La de contaminación = 0,05 con n_estimators = 200, F1-score = 1,000. Es el único punto en el que Precision y Recall valen 1 al mismo tiempo.

2. ¿La configuración con mayor F1-score cumple la restricción de 60 alertas?

Sí. Genera 50 alertas por cada 1000 conexiones, diez por debajo del máximo que el equipo puede revisar. O sea, todavía les queda un colchón.

3. ¿Qué configuración alcanzó el mayor Recall?

Todas las que tienen contamination $\geq 0,05$ llegan a Recall = 1,000. El máximo ya lo tienes en 0,05; subir a 0,06–0,12 no detecta más anomalías reales, solo infla tickets.

4. ¿Qué costo operacional produce el incremento del Recall?

En este laboratorio el Recall deja de crecer después de 0,05. Si empujas contamination para arriba no detectas más: solo suben las alertas y los falsos positivos (10 en 0,06; 30 en 0,08; 50 en 0,10; 70 en 0,12) y te pasas de la capacidad del SOC.

5. ¿Qué configuración recomiendo para el SOC y por qué?

contamination = 0,05 y n_estimators = 200. Maximiza el F1, pesca las 50 anomalías, respeta las 60 alertas y deja un margen de 10 revisiones. 0,06 solo tendría sentido si la dirección insiste en gastar todo el cupo; 0,08 o más las descarto porque saturan al equipo.

6. ¿Cuáles serían las consecuencias de subestimar contamination?

Pocas alertas y un montón de falsos negativos. Con 0,01 se te escapan 40 amenazas; con 0,03 se te escapan 20. El SOC se queda “tranquilo” mientras el incidente sigue corriendo por debajo.

7. ¿Cuáles serían las consecuencias de sobreestimarla?

Recall máximo en el papel, Precision baja y fatiga de alertas. Con 0,10 ya son 100 tickets; con 0,12, 120. Los analistas dejan de creerle al detector y se gasta más tiempo investigando normales raros que amenazas de verdad.

8. ¿Por qué el algoritmo no debería usarse como único mecanismo para bloquear conexiones?

Porque predice rareza estadística, no intención ofensiva (Chandola et al., 2009). Un proceso legítimo que mueve mucho tráfico puede salir como falso positivo; un ataque que se disfraza de tráfico habitual puede colarse como falso negativo. Si bloqueas sin analista, sin SIEM/SOAR y sin reglas de negocio, estás convirtiendo una predicción en una decisión automática y te comes toda la responsabilidad.

7. Riesgos y limitaciones

Hay que ser honestos: los datos son sintéticos y las anomalías las armé con medias bien lejos del tráfico normal. Por eso contamination = 0,05 llega a F1 = 1. En un SOC de verdad las clases se pisan más, la tasa de ataques no la conoces tan limpia y el entorno se mueve (cambian usuarios, malware y políticas). Las métricas de esta práctica son una cota optimista. El modelo tampoco te dice qué tipo de amenaza es ni te entrega una probabilidad de ataque calibrada: el score de Isolation Forest es un grado relativo de aislamiento, nada más.

Si esto se desplegara en serio, lo pondría en modo asistido (nada de bloqueo automático), con registro de la versión del modelo, monitoreo de FP/FN que confirme el analista y reentrenamiento de vez en cuando. Si no, se queda como un laboratorio bonito y poco más.

8. Conclusiones y recomendación

El ejercicio era optimizar un Isolation Forest para un SOC cuya restricción dura no es una métrica de paper, sino cuánto puede revisar la gente: 60 alertas por cada 1000 conexiones. El set tiene 950 normales y 50 anómalas. El algoritmo se entrenó sin etiquetas. Esas etiquetas solo sirvieron después para contar VP, FP, FN, Precision, Recall y F1.

Isolation Forest aísla puntos que un árbol aleatorio separa en pocos cortes (Liu et al., 2008). Contamination no estima solito la prevalencia real de ataques: fija cuántas filas cruzan el umbral y se vuelven tickets. Probar 0,01; 0,03; 0,05; 0,06; 0,08; 0,10 y 0,12 es básicamente recorrer distintos tamaños de cola para el analista. Esa es la palanca operativa del modelo.

Los resultados pintan el dilema clásico, y aquí se ve clarito porque las dos clases están bien separadas. Con contamination baja (0,01 y 0,03) la Precision es perfecta, pero el Recall vale 0,200 y 0,600: se escapan 40 y 20 anomalías. Ese es el costo de quedarse corto. En 0,05 —que coincide con la tasa real 50/1000— el modelo tira 50 alertas y, en esta corrida, 0 falsos positivos y 0 falsos negativos. Precision, Recall y F1 valen 1. A partir de 0,06 el Recall ya no mejora: se queda en 1 y cada punto extra de contamination solo fabrica falsos positivos (10, 30, 50 y 70) hasta romper el tope de 60 alertas desde 0,08. Sobreestimar contamination no “detecta más”; satura.

Como en 0,05 no había errores que mirar, revisé las configs vecinas, como dice la rúbrica. Los veinte falsos negativos de 0,03 son anomalías reales que se quedaron fuera del cupo de 30 alertas: no son tráfico normal, son las menos extremas del grupo anómalo y por eso el ranking de aislamiento las deja para el final. Los falsos positivos de 0,08 son conexiones normales de la cola estadística —un poco más de duración, bytes o puertos que el promedio de 40 s, 4500 bytes y 2 puertos— que el modelo ve “medio raras” cuando lo obligas a marcar el 8 % de las filas. El algoritmo responde a rareza, no a semántica de ataque.

La prueba con $n_estimators = 100$, dejando contamination en 0,05, no mueve el F1 de forma material frente a 200 árboles. En un problema con nubes tan distintas, el número de estimadores estabiliza el bosque; no decide cuántas alertas salen. Me quedo con $n_estimators = 200$ porque es la base reproducible del laboratorio.

Recomendación para el SOC: Isolation Forest con $n_estimators = 200$ y contamination = 0,05. Cumple la capacidad, maximiza el F1 en este conjunto y deja un margen de diez revisiones. 0,06 sería plan B solo si la dirección quiere gastar todo el cupo; no recupera amenazas extra y mete diez falsos positivos. 0,08 o más, fuera: el supuesto “incremento de Recall” ya no existe y la carga operacional se vuelve un problema.

Dos advertencias para cerrar. Primera: un F1 perfecto en datos sintéticos no te autoriza a congelar el umbral en producción. Va a haber deriva, solapamiento y etiquetas sucias. Segunda: el detector no debería bloquear conexiones solito. La predicción es un insumo para priorizar, cruzar con SIEM, EDR o intel de amenazas y decidir con alguien mirando. Una predicción no es una decisión. La solución decente junta datos de calidad, modelo evaluado, criterio de capacidad del equipo y mejora continua.

9. Referencias

- Chandola, V., Banerjee, A. y Kumar, V. (2009). Anomaly detection: A survey. *ACM Computing Surveys*, 41(3), 1-58. <https://doi.org/10.1145/1541880.1541882>
- Liu, F. T., Ting, K. M. y Zhou, Z.-H. (2008). Isolation forest. En *2008 Eighth IEEE International Conference on Data Mining* (pp. 413-422). IEEE. <https://doi.org/10.1109/ICDM.2008.17>
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M. y Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.

Nota. Las cifras de la Tabla 1 siguen el protocolo del laboratorio guiado (950 normales, 50 anomalías, $n_estimators = 200$, semilla 42) y calzan con la corrida en contamination = 0,05 (0 FP y 0 FN). Si en Colab alguna fila se corre una alerta por redondeo del parámetro, manda la tabla que salió en el cuaderno.